2024

101Alpha 因子升级及效率优化说明

应用专题-数据提取专题 08

深圳天软科技

本文档所有内容均属于深圳市天软科技开发有限公司的商业机密，深圳市天软科技开发有限公司亦保留上述文档中阐述的包括而不限于技术、方法、思想的所有权。此文档仅仅只许可因本文档所必需传播的范围内使用（仅限于内部），本文档的获得者不得将本文档散发给不相关的其他机构和个人。

深圳天软科技 | 深圳市福田区莲花街道梅岭社区新闻路 59 号深茂商业中心 19F

1 更新日志

1.1 更新日志

更新日志	更新说明
2019-07-16	文档创建和发布
2019-07-16	样式修改，修改 bug
2022-03-03	修正部分样式，增加 Stock_Alpha101_cal 说明
2024-07-01	增加 Stock_Alpha101_cal4 和 Stock_Alpha101_cal 说明，补充函数定义及参数作用，增加 FAQ

1.2 摘要

- 功能升级：
  - IndNeutralize 支持按指定行业分类层级进行中性化处理；可以通过重写 GetParm_GroupIndustry_ByEndt 自定义中性化时的行业分类
  - 规范横比缓存串：可以**适用在不同时间、不同周期、不同复权方式、不同复权基准日、不同股票池名称场景下的缓存**
- 效率优化：
  - 通过升级 101 因子底层函数，利用天软 techcachedo 技术指标缓存，提升 101 因子运算效率
  - 提升效率统计：升级前，全 A 下取一只股票的 63 因子要 4 个小时；升级之后，最耗时的为 97 因子，全 A 一天所有股票要 3 分钟，计算全 A 一天所有股票的 101 因子值只需要 34 分钟
- 101 因子计算器：Alpha_101_Cal

2 功能升级

2.1 指定行业分类标准进行中性化：IndNeutralize

➢ ***IndNeutralize 的理解***

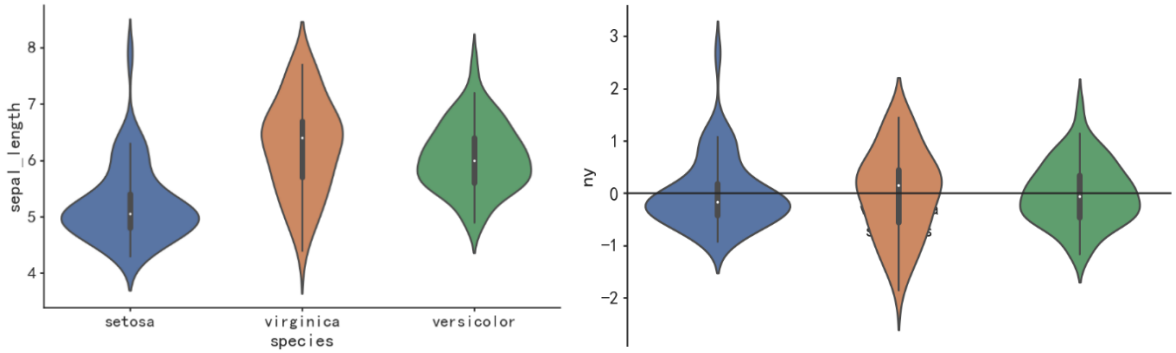
原文中对 IndNeutralize 的解释如下：

“x cross-sectionally neutralized against groups g (subindustries, industries, sectors, etc.),i.e., x is cross-sectionally demeaned within each group g”

即取得股票池截面行业分类之后，按行业分组去均值，得到与分组 g 中性化的 x。

而我们一般行业中性化的操作为：取行业分类变量生成的虚拟变量 0-1 矩阵作为自变量，取原因子与之回归之后的残差。

可以证明，两者得到得结果是一致。我们在附录中也提供了范例，可以看到两种方法进行中性化的结果相等。



图表 1：分组去均值对各组数据分布的影响

➢ ***行业分类标准的确定***

既然已经知道 IndNeutralize 主要是为了求得行业中性化因子，随之而来的一个问题是行业分类标准取什么。

原文中对行业分类数据的说明如下：

“IndClass = a generic placeholder for a binary industry classification such as GICS, BICS, NAICS, SIC, etc., in indneutralize(x, IndClass.level), where level = sector, industry, subindustry, etc.Multiple IndClass in the same alpha need not correspond to the same industry classification.”

可以看到，IndClass 仅仅是个行业分类占位符，可以参考的分类标准有 GICS、BICS、NAICS 等；而 sector, industry, subindustry 为不同的行业层级。

➢ ***我们的处理***

这里我们处理为**默认使用申万行业分类体系**，对照关系如下：（也可以通过重写 GetParm_GroupIndustry_ByEndt 自定义行业分类标准）

表格 1：101 因子公式 IndClass 对照说明

因子公式	分类	行业个数
InClass.sector	申万一级行业	28
InClass.industry	申万二级行业	104
InClass.subindustry	申万三级行业	227

此次升级**不考虑前向兼容**（在之前 2019-07-23 已发布的 101 版本中，没有考虑到行业因素，只是将所有股票看作同一个分类，然后去均值），含 **IndClass 的因子输出结果将由不同**，汇总如下：

表格 2：含 IndClass101 因子汇总

名称	名称	名称
OHO_Alpha_048	OHO_Alpha_070	OHO_Alpha_089
OHO_Alpha_058	OHO_Alpha_076	OHO_Alpha_090
OHO_Alpha_059	OHO_Alpha_079	OHO_Alpha_091
OHO_Alpha_063	OHO_Alpha_080	OHO_Alpha_093
OHO_Alpha_067	OHO_Alpha_082	OHO_Alpha_097
OHO_Alpha_069	OHO_Alpha_087	OHO_Alpha_100

2.2 取不同股票池下的因子值：Pn_StockArrName

- ***问题的提出***

101 因子计算过程中，经常涉及有这样的运算，比如：

- 计算当日横比排名：***PercentRankOftheDay***（对应 101 因子公式中的 ***Rank(x)***）
- 横截面行业中性化：***IndNeutralize***（对应 101 因子公式中的 ***indneutralize(x,g)***）
- 横截面标准化：***ScaleOfSeries***（对应 101 因子公式中 ***scale(x,a)***）

这样一些操作有一个特点即：涉及到股票池，计算某一个股票的因子值，往往需要当天所有股票参与运算。计算一只股票的因子值的排名，已经算好了，如果计算第二只股票时，又要计算全市场排名，是一件开销很大的事情。在之前的系列中，我们利用 static 缓存计算来解决这个矛盾（请在 TSL 语言帮助中，搜索”STATIC 静态计算”，了解相关内容）。

考虑到大家一个可能的需求是：

- 计算某个股票在设定不同股票池下的因子值

这带来的另一个问题是，如何正确的识别出股票池。

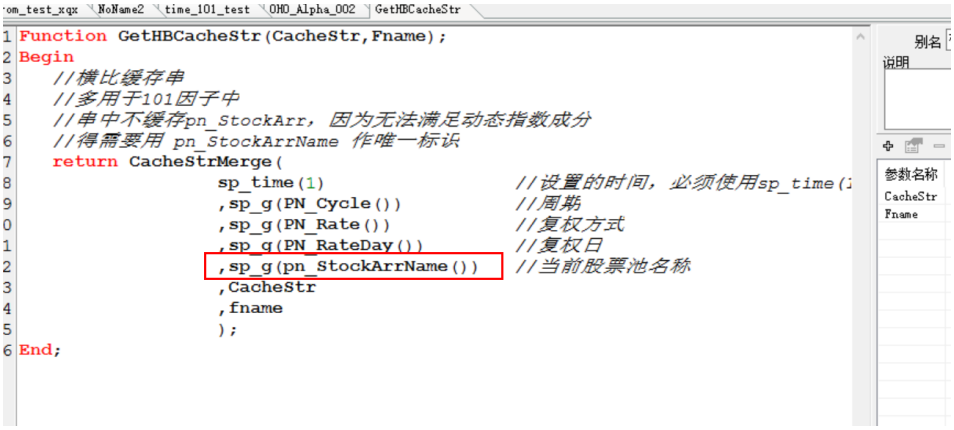
- ***开发者的解决思路***

如果利用股票池中的所有代码合并的字符串，作为唯一标识放在缓存串中，那么对于有 3000 多只股票的 A 股市场来说，这无疑会给我们带来很大的效率问题。

另一方面，考虑到另一种可能，动态指数成分，比如：101 因子中的经常操作是，计算因子排名的相

关系数，对一个指数成分来说，计算昨日股票池排名时，今昨两天的股票池本身就可能不同。

因此，我们为了更为科学合理得计算因子值，提供了模型 GetHBCacheStr。此模型专门负责横比计算中设立缓存串，可以看到我们需要提供的额外系统参数为 Pn_StockArrName，此模型用以给股票池设定唯一别名，**在计算同一股票在不同股票池下的因子值的场景中，需要用户自行指定**。



- *客户使用上的不同*

在客户使用的角度看，此处升级并没有太大的不同，只需要在***有需要区分不同股票池的场景***下，给股票池设定唯一可识别的标识即可。

以取 20190723 日浦发银行，分别在上证 50、沪深 300 当中的 OHO_Alpha_002 为例（002 中涉及到计算排名的相关系数）。

最后的代码如下：

[代码]

```
    endt := 20190723T;
   SetSysParam(PN_Stock(),'SH600000');
   SetSysParam(PN_Rate(),1);
   SetSysParam(PN_RateDay(),endt);

   StockArr := getbk('上证 50');
   SetSysParam(pn_StockArrName(),'SSZ50');
   SetSysParam(pn_StockArr(),StockArr);
   v1 := OHO_Alpha_002();
   StockArr := getbk('沪深 300');
   SetSysParam(pn_StockArrName(),'SHS300');
   SetSysParam(pn_StockArr(),StockArr);
   v2 := OHO_Alpha_002();
   return array(v1,v2);
```

[代码]

//结果（在未升级的版本中，由于静态缓存不能区分股票池，返回结果将错误的为 v1=v2）



3 效率优化

3.1 效率提升统计

此次升级，我们对计算效率也进行了大幅提升。

- ***设置股票池为全 A 的情况下，提取指定日特定股票的 101 因子值耗时***

如下，我们统计了在设置 pn_Stockarr()为全 A 时，提取一只**股票**在指定日的 101 个因子值，升级前后所耗费的时间。

可以看到，升级前需要花费 4 个小时的 063 因子，在升级后只要 48.74s 即可。升级后，最耗时的因子为 098，只需要 2 分钟。

表格 3：全 A_一只股票_101 因子耗时统计（Endt=2019-06-18T， stockID=浦发银行，StockArr=A 股）

因子名称	升级前耗时(s)	升级后耗时(s)	升级后-升级前	备注
OHO_Alpha_063	14756.49	48.74	-14707.75	63 因子
OHO_Alpha_081	2162.82	38.06	-2124.76	81 因子
OHO_Alpha_095	1543.50	29.78	-1513.72	95 因子
OHO_Alpha_098	692.54	121.80	-570.74	98 因子
OHO_Alpha_064	321.83	25.49	-296.34	64 因子
OHO_Alpha_079	283.14	47.27	-235.87	79 因子
OHO_Alpha_074	257.82	25.49	-232.33	74 因子
OHO_Alpha_099	134.64	17.66	-116.98	99 因子
OHO_Alpha_048	89.53	75.30	-14.23	48 因子
OHO_Alpha_039	79.33	29.28	-50.05	39 因子
OHO_Alpha_078	77.57	16.16	-61.41	78 因子
OHO_Alpha_062	75.09	13.78	-61.31	62 因子
OHO_Alpha_072	67.13	22.42	-44.71	72 因子
OHO_Alpha_061	50.44	26.96	-23.48	61 因子
OHO_Alpha_065	42.45	12.08	-30.37	65 因子

OHO_Alpha_087	35.13	15.19	-19.94	87 因子
OHO_Alpha_097	34.24	14.44	-19.80	97 因子
OHO_Alpha_032	33.63	30.78	-2.85	32 因子
OHO_Alpha_052	27.01	25.05	-1.96	52 因子
OHO_Alpha_019	25.68	24.04	-1.64	19 因子

➢ ***提取指定日股票池的全部成分因子值耗时***

由于我们在 101 因子计算过程中，使用了横比缓存，因此计算全部 A 股在指定日的 101 因子值，有时与只提取一只相比，并不会相差太大。

我们也统计了升级后，在上述参数设定下，提取指定日**全部 A 股**的 101 因子值所耗费的时间。

升级后，哪怕在设定股票池为全 A 的情况下，计算整个 A 股 3000 多只股票，一天所有 101 个因子值，**也能在 34 分钟，即半个小时左右完成！！！**

表格 4：指定日股票_股票池全部成分_101 因子耗时统计（Endt=2019-06-18T）

因子名称	上证 50_一天_全部	沪深 300_一天_全部	中证 1000_一天_全部	全 A_一天_全部
汇总(s)	29.935	181.697	591.381	2027.830
汇总(m)	0.497	3.018	9.823	**33.682**

我们在附件中也提供了从具体的因子角度展开的时间统计。

3.2 效率优化升级模型说明

对 101 因子的效率升级，主要在于升级基础函数，使得其能支撑技术指标缓存，然后利用 techcachedo 关键字进行加速。

对于一些计算比较复杂的 101 因子，在计算中，默认开启 techcachedo。

升级的底层函数汇总如下：

表格 5：升级 101 因子底层计算模型相关说明

模型名称	含义	修改说明	备注
AvgOfNDay	N 日平均	R_1	
CorrelOfNDay	N 日相关系数	R_1	
CovarOfNDay	N 日协方差	R_1	
HHVBarsOfNDay	N 日最高点到当前的周期数	R_1	
HighestOfNDay	N 日最高	R_1	
LLVBarsOfNDay	N 日最低点到当前的周期数	R_1	
LowestOfNDay	N 日最低	R_1	
StdOfNDay	N 日标准差	R_1	
SumOfNDay	N 日和	R_1	
ProductOfNDay	N 日乘积	R_1	
DecayAvgOfNDay	N 日线性衰减加权平均		暂不支持 techcachedo 缓存

PercentRankOfNDay	当前值在最近 N 日的分位数		暂不支持 techcachedo 缓存
IndNeutralize	中性化（无交易成分设为 NAN）	规范横比缓存串； 支持选择行业层级进行中性化	
PercentRankOftheDay	当日分位排名（无交易成分设为 NAN）	规范横比缓存串; 规范当日无交易成分设为 Nan	
RankOftheDay	当日排名（无交易成分设为 NAN）	规范横比缓存串; 规范当日无交易成分设为 Nan	
ScaleOfSeries	标准化（无交易成分设为 NAN）	规范横比缓存串	

➢ **R_1 说明：**

修改说明：参数中增加技术指标缓存串，用于支撑 techcachedo 技术指标缓存。

以 AvgOfNDay 为例，修改后函数原型为:

- AvgOfNDay(Exp,N,CacheStr:String):Real

其中，CacheStr 用于对表达式 Exp 建立**字符串唯一标识**，当不指定 CacheStr 时，则不会进行指标缓存。**如果缓存串不能将表达式正确分开，在 techcachedo 将使得结果出现异常**。

```
techcachedo
  begin
   sp_s(PN_Stock(),'SH000300');
   sp_s(PN_Date(),20190725T);
   return array(
            AvgOfNDay(@open(),100,'test'),
            AvgOfNDay(@close(),100,'test')//不同表达式设定同一缓存串，结果异常
            );
  End
```

以附录中的代码为例测试效率提升情况，可以看到，添加缓存串之后开启 techcachedo 只需要 0.029s 即可计算完成之前需要花费 0.6s 的计算工作。

场景	耗时(s)
未开技术指标缓存	0.601
技术指标缓存+未加缓存串	0.603
技术指标缓存+正确的缓存串	0.029

➢ **规范横比缓存串：**

之前的横比缓存串如下：

```
Function PercentRankOftheDay(Exp, CacheStr);
Begin
{
  股票横截面数据排名，x表示一组股票的截面数据，按向量x的元素大小升序排序，序号从0开始，
  如[0,1,...,n]，然后输出排名所占总位数的百分比，即将排名归一到[0,1]区间。
  Nan值不参与排名，输出Nan。
  例如：输入原始x向量，x=[6,4,6,2,9,9]，升序排名数组r=[2,1,2,0,4,4]，再将r/(length(x)-1)
        归一化后输出数组[0.4,0.2,0.4,0,0.8,0.8]。
        若含有nan，x=[1,nan,2,5]，输出[0,nan,0.5,1]

  加一个参数CacheStr解决缓存-2018-08-31

}

  t:=static GetDataSub(Exp) name functionname()+CacheStr+datetostr(sp_time());
  StockId:=DefaultStockID();
```

可以看到，只控制了时间因素

规范化之后的横比缓存串，统一由模型 GetHBCacheStr 控制，可以**适用在不同时间、不同周期、不同复权方式、不同复权基准日、不同股票池名称场景下的缓存**。

```
demo_file_xqx  findusrfun  GetHBCacheStr  PercentRankOftheDay2

    输入：
        无
    输出：
        无
    更新日志：
       1、加一个参数CacheStr解决缓存-2018-08-31
       2、横比，缓存中必须加sp_time(1)，而不是sp_time()


    }
    Cstr := GetExpCaceStr(exp, CacheStr);
    if Cstr=False then
       t := GetDataSub(Exp);
    else
    begin //缓存
       t:=static GetDataSub(Exp,Cstr)
             name GetHBCacheStr(Cstr,FunctionName());
    end
```

```
GetHBCacheStr

Function GetHBCacheStr(CacheStr,Fname);
Begin
   //横比缓存串
   //多用于101因子中
   //串中不缓存pn_StockArr，因为无法满足动态指数成分
   //得需要用 pn_StockArrName 作唯一标识
   return CacheStrMerge(
                  sp_time(1)                  //设置的时间，必须使用sp_time(1)
                  ,sp_g(PN_Cycle())           //周期
                  ,sp_g(PN_Rate())            //复权方式
                  ,sp_g(PN_RateDay())         //复权日
                  ,sp_g(pn_StockArrName())    //当前股票池名称
                  ,CacheStr
                  ,fname
                  );
End;
```

4 101 因子计算模型

4.1 模型汇总

基于 101 因子的提取过程，封装了高级接口，汇总如下：

二级分类	是否高级接口	名称	别名	是否实时计算	备注
个券	否	OHO_Alpha_***(001->101)	第 1-101 个因子	是	基础模型；不推荐直接使用
	是	Stock_Alpha101_Cal4	指定日 101 因子值		
		Stock_Alpha101_Cal	区间每日 101 因子值		
		Stock_Alpha101_Cal3	最新日 101 因子值		
整个板块/指数成分		Alpha_101_Cal	101 因子计算器		

4.2 Alpha_101_Cal：101 因子计算器

4.2.1 函数说明

- 定义：

  Alpha_101_Cal(Alphas,TargetStocks,Begt,EndT,StockArr,StockArrfunc,StockArrName,ifDynamic,ifpost):Array
- 说明：101 因子计算器，与周期，复权方式，复权基准日相关
- 参数：

  Alphas :1-101 之间的整数或整数数组，表指定要计算的 101 因子，为 nil 或空数组时表计算全部 101 个因子，比如：

```
//表计算 OHO_Alpha_001
alphas := 1;
//表分别计算 OHO_Alpha_001,OHO_Alpha_002
alphas := array(1,2);
```

  TargetStocks：数组，需要返回因子值的股票，为 nil 或空数组时表取所有成分

  Begt:日期，区间开始日

  EndT:日期，区间截止日

  StockArr:一维字符串数组，用于计算横比的股票池，StockArr 与 StockArrfunc 二选一指定即可

  StockArrfunc：求解每日股票池的函数，接受参数 day，比如：

```
StockArrfunc := function (endt);
  begin
    return GetBKByDate('SH000300',EndT);
  end
```

  StockArr 与 StockArrfunc 二选一指定即可

StockArrName：字符串，必须指定，唯一的股票池名称，如果为 nil 或""时，根据当前时间生成唯一串,并通过 **StockArrName** 返回

ifDynamic :布尔值，是否动态指数成分(只在给定 StockArrfunc 时有效)， **默认为 False**

动态指数成分的提出背景如下：

以 OHO_Alpha_003 为例，需要计算横比排名相关系数，

20190725T 计算 SH600000 在过去 6 天，每天开盘价

在指数 SH000300 成分中横比排名时，

可以选择：

(1) 在 20190725T 的 sh000300 成分往前推 6 天

(2) 也可以往前推 6 天，每天取 sh000300 的实际成分，【此时 ifDynamic=True】

ifpost：是否动态复权，动态复权即将循环日期设置为复权基准日，**默认为 True**

比如需要计算 20190701T->20190730T 的每天的因子值，动态的将复权基准日设置为当天。

- 返回：

字段	类型	单位	非空	备注
截止日	Date		是	
代码	String		是	
名称	String		是	
OHO_Alpha_001	Numeric		是	由参数 **Alphas** 选择的 101 因子
OHO_Alpha_002	Numeric		是	
……				

4.2.2 范例一：固定每日成分

```
//根据板块计算因子值，注：默认动态复权，自定义板块名
  sp_s(PN_Rate(),1);
  sp_s(pn_cycle(),cy_day());
   Alphas := array(101,100);
   TargetStocks := array();//为空表取所有
   begt := 20180101T;
   endt := 20190201T;
   StockArr := getbk('沪深 300');
  return Alpha_101_Cal(Alphas,TargetStocks,Begt,EndT,StockArr);
```

4.2.3 范例二：自定义每日成分

```
//指定指数计算因子值，注：默认动态复权，自定义板块名
  sp_s(PN_Rate(),1);
  sp_s(pn_cycle(),cy_day());
   Alphas := array(101,100);
```

```
TargetStocks := array();//为空表取所有
begt := 20180101T;
endt := 20190201T;
StockArr := nil; //不指定股票池，通过 stockarrfunc 取
stockarrfunc := function (endt);
begin
  return GetBKByDate('SH000300',300);
end;
return Alpha_101_Cal(Alphas,TargetStocks,Begt,EndT,
        StockArr,stockarrfunc
        );
```

4.3 Stock_Alpha101_cal

101 因子计算的一步抽象，以上都基于 101 因子计算器 Alpha_101_Cal。从使用的角度看以接口形式统一股票池的设置，以及其他技术细节处理，方便准确快速的取数。

汇总如下：

名称	别名	函数原型
Stock_Alpha101_Cal4	获取当前个股的指定日 101 因子值	Stock_Alpha101_Cal4(Alphas,Endt,**IndexorBk,ifDynamic,ifpost**):number;
Stock_Alpha101_Cal	获取当前个股的区间每日 101 因子值	Stock_Alpha101_Cal(Alphas,Begt,EndT,**IndexorBk,ifDynamic,ifpost**):table;
Stock_Alpha101_Cal3	获取当前个股的当前日 101 因子值	Stock_Alpha101_Cal3(Alphas,**IndexorBk,ifDynamic,ifpost**):number;

提供汇总使用范例如下：

```
[code]
//指数 SH000300 中的浦发银行，在 20191112，1 号 101 因子值
   SetSysParam(PN_Stock(),'SH600000');
  SetSysParam(PN_Date(),20191112T);
     //区间
  return Stock_Alpha101_Cal(1,20191101T,20191112T,'SH000300');
      //当前日
  return Stock_Alpha101_Cal3(1,'SH000300');
     //指定日
  return Stock_Alpha101_Cal4(1,20191111T,'SH000300');
[code]
```

4.3.1 Stock_Alpha101_Cal3：获取当前个股的当前日 101alpha

- 定义：Stock_Alpha101_Cal3(Alphas,IndexorBk,ifDynamic,ifpost)

- 说明：获取当前个股的当前日 101 因子值
- 参数：

Alphas :1-101 之间的整数或整数数组，表指定要计算的 101 因子，为 nil 或空数组时表计算全部 101 个因子，比如：

```
//表计算 OHO_Alpha_001
alphas := 1;
//表分别计算 OHO_Alpha_001,OHO_Alpha_002
alphas := array(1,2);
```

IndexorBk：板块名或者指数名，用来在成分中排序，比如：”沪深 300”，可以取’SH000300’

ifDynamic :布尔值，是否动态指数成分(只在给定 **IndexorBk 为指数代码**时有效)， **默认为 False**

动态指数成分的提出背景如下：

以 OHO_Alpha_003 为例，需要计算横比排名相关系数，

20190725T 计算 SH600000 在过去 6 天，每天开盘价

在指数 SH000300 成分中横比排名时，

可以选择：

(1) 在 20190725T 的 sh000300 成分往前推 6 天

(2) 也可以往前推 6 天，每天取 sh000300 的实际成分，【此时 ifDynamic=True】

ifpost : 是否动态复权，动态复权即将循环日期设置为复权基准日，**默认为 True**

比如需要计算 20190701T->20190730T 的每天的因子值，动态的将复权基准日设置为当天。

- 范例：

```
//板块沪深 300 中的浦发银行，在 20191112，1 号 101 因子值
SetSysParam(PN_Stock(),'SH600000');
SetSysParam(PN_Date(),20191112T);
return Stock_Alpha101_Cal3(1,'沪深 300');//-0.0475
```

4.3.2 Stock_Alpha101_Cal4：获取当前个股的指定日 101alpha

- 定义：Stock_Alpha101_Cal4(Alphas,Endt,IndexorBk,ifDynamic,ifpost)
- 说明：获取当前个股的指定日 101 因子值
- 参数：

Endt :指定日，日期型日期

其他参数同 Stock_Alpha101_Cal3

- 范例：

```
//板块沪深 300 中的浦发银行，在 20191112，1 号 101 因子值
SetSysParam(PN_Stock(),'SH600000');
return Stock_Alpha101_Cal4(1,20191112T,'沪深 300');//-0.0475
```

4.3.3 Stock_Alpha101_Cal：获取当前个股的区间每日 101 因子值

- 定义：Stock_Alpha101_Cal(Alphas,Begt,EndT,IndexorBk,ifDynamic,ifpost)
- 说明：获取当前个股的区间每日 101 因子值
- 参数：

  Begt :开始日，日期型日期

  Endt :截止日，日期型日期

  其他参数同 Stock_Alpha101_Cal3
- 范例：

```
//板块沪深 300 中的浦发银行，从 20191101 到 20191112 每天，1 号 101 因子值
SetSysParam(PN_Stock(),'SH600000');
return Stock_Alpha101_Cal(1,20191101T,20191112T,'沪深 300');
//返回结果：
```

	截止日	代码	名称	OHO_Alpha_001
0	43,770.00000000000000000	SH600000	浦发银行	0.02290076335877870
1	43,773.00000000000000000	SH600000	浦发银行	0.46564885496183200
2	43,774.00000000000000000	SH600000	浦发银行	-0.50000000000000000
3	43,775.00000000000000000	SH600000	浦发银行	-0.23664122137404600
4	43,776.00000000000000000	SH600000	浦发银行	-0.09315589353612170
5	43,777.00000000000000000	SH600000	浦发银行	-0.00570342205323193
6	43,780.00000000000000000	SH600000	浦发银行	0.14258555133079900
7	43,781.00000000000000000	SH600000	浦发银行	-0.04752851711026620

5 FAQ

5.1 Q：101 因子是否适用于高频环境?

参照 101 因子原文的理解，我们**认为 101 因子还是属于中低频**，也就是适于日线或日线更低频。但是就我们封装的 101 因子公式来说，并没有在内部限定计算时的频率。

比如：在计算 OHO_Alpha_003 在设置为分钟线时，其内部提取的收盘价与成交量都是分钟线对应的数据。

也就是说，在技术上时可行的。

但是， **OHO_Alpha_056 除外**，内部使用了 stocktotalvalue3()，提取当日最新市值，只支持日线及更低频率，导致，在设置分钟线时，取 stocktotalvalue3()，将取得以当日收盘价计算的市值，即可能有取到未

来数据的风险。

因此，除 **OHO_Alpha_056 外，**101 因子都是可以正确的在高频环境取得数据。

5.2 Q：升级之后，为什么同样的代码有的因子计算结果有差异?

由于此次升级考虑了分行业中性化，如果所选因子包含行业中性化算子即：IndNeutralize，则该因子计算结果将会有较大的差异。

请参考汇总的受影响因子列表——表格 2：含 IndClass101 因子汇总。

如果想去掉此次升级的影响，请重写 GetParm_GroupIndustry_ByEndt。

5.3 Q：如何获得组合成分股的 101 因子值

Stock_Alpha101_Cal、Stock_Alpha101_Cal3、Stock_Alpha101_Cal4 三个模型只负责计算当前个股的情况，不支持计算后取出其组合券的场景。获得指定组合成分股所有情况需采用 Alpha_101_Cal。

6 附录

6.1 对已有数据进行 IndNeutralize 的两种方式——代码范例

```
endt := 20190716T;
Stocks := GetBKByDate('SH000300',endt);
SetSysParam(pn_StockArr(),Stocks);
data := query('',array2str(Stocks),True,'代码',
            '代码',DefaultStockID(),
            'PE',StockPE(endt),
            'PE2',StockPE(endt),
            '行业',StockSWIndustryIdLv1());
   //由分类变量生成指示虚拟变量
  dummy_mat := get_dummies(data[:,'行业']);
   //残差因子 u
  Regress_CMLS(data[:,'PE'],dummy_mat,u,false);
   //分组去均值因子 u2
  u2 := GroupDemeanIt(data,'行业','PE');
  return u | u2 |(u - u2);//u 与 u2 一致
```

6.2 自定义 InClass 行业分类——代码范例

```
//备注：受目前数据的限制，中证行业分类，只能取最新的，不能回溯
Function GetParm_GroupIndustry_ByEndt(t,Endt,ByGroup);
```

```
Begin
   {
   ByGroup := 0; //强制不分行业，影响模型 IndNeutralize
   }
   r := array();
   if not IsTable(StockArr) then
      return r;
   if '代码' in mcols(StockArr,1) then
      r := t;
   else
      r := `array('代码':t);

   with *,array(pn_date():Endt) do
   begin
      if ByGroup=1 then
      begin
         update r set
            ["行业名称"]=spec(base(10032),["代码"]),
            ["行业"]=spec(base(10042),["代码"])
            end;
      end
      else if ByGroup=2 then
      begin
         update r set
            ["行业名称"]=spec(base(10033),["代码"]),
            ["行业"]=spec(base(10043),["代码"])
            end;
      end
      else if ByGroup=3 then
      begin
         update r set
            ["行业名称"]=spec(base(10033),["代码"]),
            ["行业"]=spec(base(10044),["代码"])
            end;
      end
      else
         r[:,'行业'] := '全市场';
   end
   return r;
End;
```

6.3 AvgofNday 技术指标缓存效率测试

```
function fun1();
begin
   ret := array();
   for i := 1 to 3 do
   begin
     mtic;
     r := avgcaltype(i);
     t := mtoc;
     ret[i] := array('类型':i,'耗时':t,'结果':r);
   end
   return ret;
end;
function avgcaltype(types);
begin
   n := 100;
   days := nday3(n,sp_time());
   case types of
      1: techcachedo
         begin
            //开启技术指标缓存
            days::=specdate(AvgOfNDay(@StockZf3(),250,'stockzf1'),mcell);
         end
      2: techcachedo
         begin
            //开启技术指标缓存，未给表达式加字符串缓存串,缓存失效
            days::=specdate(AvgOfNDay(@StockZf3(),250),mcell);
         end
      3: begin
         //正常版本
         days::=specdate(AvgOfNDay(@StockZf3(),250),mcell);
      end
   end;
   return days;
end
```

6.4 提取一天所有 A 股的 101 因子值耗时详情

因子名称	上证 50_一天_全部	沪深 300_一天_全部	中证 1000_一天_全部	全 A_一天_全部	耗时排名(从大到小)	备注
OHO_Alpha_001	0.097	0.628	2.017	6.884	50	1 因子

OHO_Alpha_002	0.031	0.194	0.652	2.018	69	2 因子
OHO_Alpha_003	0.039	0.236	0.791	2.880	65	3 因子
OHO_Alpha_004	0.020	0.122	0.404	1.487	74	4 因子
OHO_Alpha_005	0.013	0.080	0.271	0.971	81	5 因子
OHO_Alpha_006	0.006	0.037	0.124	0.451	92	6 因子
OHO_Alpha_007	0.086	0.525	1.797	3.089	64	7 因子
OHO_Alpha_008	0.027	0.161	0.554	1.915	70	8 因子
OHO_Alpha_009	0.018	0.108	0.371	0.722	86	9 因子
OHO_Alpha_010	0.050	0.307	1.068	2.763	67	10 因子
OHO_Alpha_011	0.015	0.089	0.294	1.024	78	11 因子
OHO_Alpha_012	0.004	0.024	0.081	0.131	97	12 因子
OHO_Alpha_013	0.023	0.135	0.451	1.671	72	13 因子
OHO_Alpha_014	0.012	0.070	0.234	0.812	84	14 因子
OHO_Alpha_015	0.045	0.268	0.902	3.305	63	15 因子
OHO_Alpha_016	0.023	0.135	0.454	1.683	71	16 因子
OHO_Alpha_017	0.054	0.330	1.130	3.780	60	17 因子
OHO_Alpha_018	0.011	0.065	0.214	0.787	85	18 因子
OHO_Alpha_019	0.327	2.039	6.867	24.195	28	19 因子
OHO_Alpha_020	0.009	0.052	0.172	0.654	88	20 因子
OHO_Alpha_021	0.021	0.127	0.430	1.554	73	21 因子
OHO_Alpha_022	0.017	0.101	0.343	1.171	76	22 因子
OHO_Alpha_023	0.007	0.038	0.123	0.440	93	23 因子
OHO_Alpha_024	0.041	0.254	0.845	2.825	66	24 因子
OHO_Alpha_025	0.009	0.057	0.194	0.680	87	25 因子
OHO_Alpha_026	0.091	0.554	1.793	6.389	52	26 因子
OHO_Alpha_027	0.052	0.314	1.030	3.782	59	27 因子
OHO_Alpha_028	0.049	0.306	1.019	3.538	61	28 因子
OHO_Alpha_029	0.143	0.866	2.934	10.090	43	29 因子
OHO_Alpha_030	0.019	0.115	0.377	1.352	75	30 因子
OHO_Alpha_031	0.160	0.987	3.372	11.130	42	31 因子
OHO_Alpha_032	0.412	2.532	8.560	30.002	21	32 因子
OHO_Alpha_033	0.003	0.015	0.047	0.177	96	33 因子

OHO_Alpha_034	0.029	0.178	0.590	2.052	68	34 因子
OHO_Alpha_035	0.062	0.384	1.295	4.583	56	35 因子
OHO_Alpha_036	0.200	1.208	4.168	14.563	36	36 因子
OHO_Alpha_037	0.257	1.563	5.385	18.749	32	37 因子
OHO_Alpha_038	0.008	0.049	0.167	0.602	89	38 因子
OHO_Alpha_039	0.386	2.338	8.043	28.215	23	39 因子
OHO_Alpha_040	0.012	0.070	0.237	0.836	83	40 因子
OHO_Alpha_041	0.001	0.007	0.024	0.085	99	41 因子
OHO_Alpha_042	0.005	0.030	0.100	0.360	94	42 因子
OHO_Alpha_043	0.115	0.707	2.388	7.998	47	43 因子
OHO_Alpha_044	0.014	0.081	0.271	0.990	80	44 因子
OHO_Alpha_045	0.051	0.317	1.062	3.784	58	45 因子
OHO_Alpha_046	0.005	0.029	0.097	0.335	95	46 因子
OHO_Alpha_047	0.016	0.097	0.329	1.165	77	47 因子
OHO_Alpha_048	2.380	14.886	49.802	115.161	3	48 因子
OHO_Alpha_049	0.007	0.041	0.135	0.464	90	49 因子
OHO_Alpha_050	0.103	0.632	2.104	7.613	48	50 因子
OHO_Alpha_051	0.006	0.041	0.128	0.453	91	51 因子
OHO_Alpha_052	0.349	2.222	7.215	24.871	25	52 因子
OHO_Alpha_053	0.002	0.012	0.039	0.087	98	53 因子
OHO_Alpha_054	0.001	0.005	0.015	0.054	100	54 因子
OHO_Alpha_055	0.089	0.551	1.824	6.491	51	55 因子
OHO_Alpha_056	0.071	0.402	1.259	4.345	57	56 因子
OHO_Alpha_057	0.046	0.281	0.947	3.309	62	57 因子
OHO_Alpha_058	0.302	1.888	6.232	22.135	30	58 因子
OHO_Alpha_059	0.517	3.231	10.784	37.870	15	59 因子
OHO_Alpha_060	0.012	0.075	0.247	0.900	82	60 因子
OHO_Alpha_061	0.343	2.150	7.307	25.282	24	61 因子
OHO_Alpha_062	0.182	1.161	3.822	13.506	37	62 因子
OHO_Alpha_063	0.612	3.900	12.564	44.188	13	63 因子
OHO_Alpha_064	0.344	2.144	6.889	24.793	26	64 因子
OHO_Alpha_065	0.167	1.039	3.342	12.139	39	65 因子

OHO_Alpha_066	0.082	0.517	1.663	5.601	54	66 因子
OHO_Alpha_067	0.079	0.493	1.583	5.725	53	67 因子
OHO_Alpha_068	0.413	2.578	8.425	30.459	18	68 因子
OHO_Alpha_069	0.171	1.044	3.396	11.974	40	69 因子
OHO_Alpha_070	0.414	2.538	8.203	29.482	22	70 因子
OHO_Alpha_071	1.849	11.687	38.503	138.233	2	71 因子
OHO_Alpha_072	0.298	1.886	6.098	21.988	31	72 因子
OHO_Alpha_073	0.101	0.641	2.061	4.713	55	73 因子
OHO_Alpha_074	0.327	2.063	6.690	24.170	29	74 因子
OHO_Alpha_075	0.161	1.003	3.209	11.718	41	75 因子
OHO_Alpha_076	1.584	9.936	31.787	113.127	6	76 因子
OHO_Alpha_077	0.123	0.811	2.539	9.162	45	77 因子
OHO_Alpha_078	0.210	1.355	4.283	15.559	35	78 因子
OHO_Alpha_079	0.610	3.908	12.454	44.763	12	79 因子
OHO_Alpha_080	0.097	0.628	1.980	7.002	49	80 因子
OHO_Alpha_081	0.514	3.259	10.372	37.635	16	81 因子
OHO_Alpha_082	0.457	2.860	9.149	32.190	17	82 因子
OHO_Alpha_083	0.014	0.084	0.273	0.996	79	83 因子
OHO_Alpha_084	0.235	1.497	4.768	17.088	34	84 因子
OHO_Alpha_085	0.176	1.138	3.569	12.890	38	85 因子
OHO_Alpha_086	0.347	2.192	6.912	24.677	27	86 因子
OHO_Alpha_087	0.542	3.443	10.872	39.118	14	87 因子
OHO_Alpha_088	1.214	7.100	22.249	79.148	7	88 因子
OHO_Alpha_089	0.847	5.149	16.643	50.382	11	89 因子
OHO_Alpha_090	0.130	0.789	2.533	9.172	44	90 因子
OHO_Alpha_091	0.808	4.924	15.960	57.766	10	91 因子
OHO_Alpha_092	0.980	2.796	8.323	30.068	20	92 因子
OHO_Alpha_093	0.912	5.472	17.895	63.603	9	93 因子
OHO_Alpha_094	0.925	5.620	18.257	65.073	8	94 因子
OHO_Alpha_095	0.422	2.555	8.317	30.139	19	95 因子
OHO_Alpha_096	1.682	10.077	32.922	117.735	5	96 因子
OHO_Alpha_097	2.541	15.425	49.966	176.955	1	97 因子

OHO_Alpha_098	1.670	10.292	33.487	120.718	4	98 因子
OHO_Alpha_099	0.250	1.561	4.885	17.648	33	99 因子
OHO_Alpha_100	0.124	0.784	2.413	8.766	46	100 因子
OHO_Alpha_101	0.001	0.004	0.014	0.050	101	101 因子
汇总(秒)	**29.935**	**181.697**	**591.381**	**2027.830**		
汇总(分)	**0.497**	**3.018**	**9.823**	**33.682**		